ChatGPT

React Scheduling App Code Review Agent Prompt

agent.md (Project Context and Agent Instructions)

- **Project description:** This is a **React**-based web application for scheduling subcontracted workers. It includes features to create and customize shift schedules, optimize staffing based on demand (e.g. derived from sales data), and allow user interactions for adjusting and recording shifts. The app uses modern web technologies (React hooks, JSX, CSS) and currently integrates with **Supabase** for backend data (though not fully enabled yet).
- **Scope:** The agent should act as a senior web developer/code reviewer with deep React expertise. It must **analyze the entire codebase**, understand component structure and data flows, and ensure all specifications and features (as implied by the project instructions above) are implemented correctly. The agent must follow the steps below (Overview, Issues, etc.), focusing on React/JS/HTML/CSS specifics rather than Python/Tkinter.
- **Technologies to consider:** Latest stable **React** (with Hooks and functional components), **Node.js**/npm environment, any UI libraries or CSS frameworks in use (e.g. Tailwind, Material-UI, or custom CSS), and **Supabase** integration. Ensure Node and React versions are compatible, dependencies are up-to-date, and linting/config tools (ESLint, Prettier) are configured [1] [2]. Include project config (like `.env` for API keys) and verify no secrets or hard-coded credentials are exposed.
- **Agent behavior tips:** Provide clear, focused guidance. Narrow analysis to relevant files (using greppable identifiers like component names) [3]. Verify all suggestions by running static checks, linters or tests [4]. Break large tasks into smaller steps for clarity [5]. The agent should not assume code is correct; it must critically test and justify every change with best practices and citations. Emphasize readability, maintainability, and user-friendly UI/UX in all recommendations.
- **Vibe-coding caution:** Because the app's code may have been partially generated with AI, be vigilant for issues common in "vibe-coded" apps [6] [7]. AI-generated code often introduces **security vulnerabilities** (e.g. XSS, SQL injection) and maintainability problems [8] [9]. Explicitly check for unsanitized inputs, improper data handling, and undocumented logic. Insist on sanitization (e.g. using DOMPurify or React safe methods) to prevent XSS [9], and validate all external data. Ensure code is well-structured and documented, since AI outputs tend to be unstructured and hard to debug [6] [10].
- **Prompting best practices:** Follow OpenAI's Codex prompting guidelines [3] [4]. Use **clear instructions** for each step, verify outputs (by suggesting linting/testing), and include environment setup if needed. For example, remind to run `npm run lint` or `npm test` after changes. Ask the agent to confirm changes with example tests or manual checks.

Codebase Overview

- **Component structure:** Identify all major React components and how they relate. Summarize the component hierarchy (e.g. top-level pages, navigation, forms, tables). For instance, list components such as **ScheduleForm**, **ShiftList**, **WorkerTable**, **DemandChart**, etc., and note if they use React Router for navigation or context providers for global state. Highlight entry points (e.g. `index.js` mounting `<App />`), and note any container vs. presentational components.

- **Key modules:** Inventory of all code files, grouping by purpose (e.g. **UI components**, **form components**, **state management**, **API services**). Note if custom hooks or utility modules exist (for scheduling logic, date handling, API calls). Check for a **API client** (e.g. Supabase client) and configuration files (`.env`, config files) to understand data flow.
- **Technologies and versions:** Record the React version and any important libraries (e.g. `react-router`, state management like Redux or Context API, charting libs, date libraries like `date-fns` / `moment`, UI frameworks, etc.). Verify Node/npm versions via `.nvmrc` or `package.json` [1]. Confirm presence of linters (ESLint), formatter (Prettier), and test frameworks (Jest, React Testing Library) as these are crucial for review [2] [11].
- **App flow:** Describe the user flow: what happens on app load, how users create or view schedules, how the app fetches or updates data. Note important routes or pages (e.g. `/schedule`, `/workers`, etc.) and key UI controls (buttons, forms). For example, if there is a "Create Shift" form, note its fields and validation method. Document any real-time features (subscriptions) or background jobs (optimization routines) if present.

Identify Issues and Bugs

- **React pitfalls:** Look for common React issues: missing **key** props in list `.map()`s (can cause rendering bugs) [12]; stale state or effects (e.g. not using `useEffect` cleanup leading to memory leaks); unnecessary state updates causing infinite loops. Check that effects have proper dependency arrays to avoid extraneous fetches or console errors. Verify that controlled components (inputs) use state properly (avoid uncontrolled inputs triggering React warnings). Ensure only one root React render call (multiple roots can cause instability).
- **UI responsiveness:** Check for blocking code (heavy loops, synchronous computations) that could freeze the UI. Ensure data fetching and heavy calculations (like schedule optimization) are async or offloaded (e.g. use Web Workers or optimized algorithms). Look for timeouts/intervals used correctly (cleared on unmount). Ensure UI layout uses responsive design (CSS grid/flex, media queries) so forms and tables display well on different screen sizes.
- **Security vulnerabilities:** Scan for XSS and injection risks. Any rendering of user-generated HTML via `dangerouslySetInnerHTML` should be sanitized [9]. Verify inputs are validated/sanitized before sending to backend or displaying. Check API calls or SQL queries for parameterization to prevent injection (e.g. ensure Supabase queries use safe practices). Ensure no sensitive data (API keys, credentials) is exposed in client code or logs [9] [8].
- **General bugs:** Watch for syntax errors or typos (lint can catch these). Check promises and async/await usage – unhandled promise rejections or forgotten `await` can cause silent failures. Ensure error cases are handled (network errors, null returns). Verify that dynamic imports or lazy-loading do not break (if used, confirm they have fallback UIs). For example, missing `<Suspense>` around lazy components might lead to runtime errors.
- **Performance:** Identify hot paths: large list renders without pagination or virtualization (see [18†L1142-L1150]). Repeated calculations in render can slow UI; suggest `useMemo` / `useCallback` to memoize heavy work. Remove unused libraries or code to reduce bundle size [13]. Check image/media sizes – ensure lazy loading or optimized formats (e.g. WebP) are used for performance [14].
- **Type and structure issues:** If TypeScript or PropTypes are used, ensure typings are correct. Otherwise, look for inconsistent data shapes (e.g. missing keys on objects). Ensure component props are validated (PropTypes) or self-documented via JSDoc.
- **Categorization:** Organize issues by type: *UI/UX*, *performance*, *data handling*, *security*, *maintainability*. This helps prioritize fixes.

Feature and Functionality Verification

- **Requirements compliance:** Compare implemented features to the project's goals: scheduling and shift optimization. Check that all required fields and actions exist. For example, if the app should "allow entering worker availability," ensure there's an interface for that. Verify that demand data (from sales) is input or imported correctly and affects scheduling output. Highlight any missing features (e.g. if no "download schedule" option is present but was required).
- **Forms and inputs:** Ensure all user inputs (dates, times, sales figures, worker details) have proper validation. Required fields should prevent empty submissions (using HTML5 validation or custom logic). For instance, an email field should use `type="email"` or regex check. Error messages should be displayed clearly (not via `alert`, but near the field, or as a visible banner). Missing or incorrect validation is a functional gap.
- **Interactions and logic:** Test form submission flows in mind: does clicking "Submit" trigger the right API call? Are loading indicators shown while saving? Are success or error messages displayed and cleared properly? For example, if a schedule save fails, the app should catch the error and inform the user instead of failing silently. Also verify that any scheduling algorithm runs as expected (simulate by giving input). If there are features like "optimize schedule", ensure they execute and update UI.
- **User stories:** Derive typical user stories (e.g., "As a manager, I can assign a shift to a worker and save the schedule.") and confirm each is fulfilled. If the instruction says "optimize shifts based on demand," check that logic is implemented (perhaps by comparing schedules before/after optimization). Document any deviation: e.g., "Customer feedback form is missing" or "Shift swap feature is incomplete."
- **Edge cases:** Consider empty states (no workers, no sales data) – the UI should handle these gracefully (e.g. show "No data" messages instead of breaking). Check boundary conditions (end-of-day time, shifts spanning midnight, etc.). If certain features (like exporting schedules) were implied, ensure they exist or note them missing.

Integration and User Interaction Checks

- **Component integration:** Verify that components communicate correctly. For example, ensure a **ScheduleForm** component properly sends data to the parent or to global state, and that parent passes updated props to a **ShiftList**. Check that props drilling or Context usage is consistent. If Redux or Context API is used, ensure state slices are updated and consumed correctly.
- **Data flow:** Trace a typical data flow: a user enters info → component state (or form library) → event handler → API call (Supabase) → response → state update → UI refresh. Ensure each step is connected (no missing handlers, no broken promise chains). For instance, if a data fetch happens on mount (`useEffect`), check that it updates state and triggers a re-render with fetched data. Look for missing dependencies or cleanup.
- **Styling and layout:** Check for consistent styling across the app. If a UI library or global stylesheet is used, ensure components adhere to theme (e.g., consistent colors, fonts, spacing). Inspect layout management (use of CSS Grid/Flexbox) to ensure forms and tables align neatly. If any `className` or inline style is used, ensure it matches design guidelines (e.g. visible focus outlines for accessibility).
- **Event bindings:** Confirm that all interactive elements have handlers. Buttons should have `onClick`, inputs should have `onChange` or `onInput`. Keyboard accessibility should be supported (e.g. form submit on Enter key). If custom dropdowns or modals exist, ensure they close on outside click or Escape key. For example, a missing `onKeyPress` or incorrect handler can break user flow.

- **Routing and links:** If React Router is used, ensure `<Link>` or `<NavLink>` components navigate correctly, and that routes match components. Check that 404 or fallback routes are handled. For example, if a URL `/schedule` should show the schedule page but shows blank, that's a link issue.
- **Backend integration:** Review how Supabase (or API) calls are made. Ensure endpoints and auth are correct. If Supabase RLS (Row Level Security) is planned but not implemented, note this in comments. Check that error handling wraps API calls (e.g. `try/catch`) and that tokens/credentials are not exposed in code.
- **Consistent user experience:** All pages should have a unified header/footer or navigation bar. Buttons and form fields should use consistent styles (size, padding). Confirm that interactive elements show hover/focus states. Ensure mobile users see a usable layout (responsive design).
- **Cross-module consistency:** For example, date formatting should be uniform across components (using the same helper or locale). Shared state (like logged-in user info) should be accessed consistently via a store or context, not mismatched local storage usage.

Redundancy and Optimization

- **Duplicate code:** Identify repeated blocks (e.g. form fields coded multiple times). Suggest refactoring by extracting reusable components or hooks. For instance, if multiple forms have similar input validation, create a shared validation hook or component. Duplicate CSS classes should be consolidated (use a CSS module or theme variables). Enforce DRY (Don't Repeat Yourself) principle.
- **Inefficient patterns:** Look for code in render paths that can be optimized. For example, avoid doing heavy computations inside JSX; instead compute outside or memoize. If loops (like `.map()`) are within render, ensure they're not recalculating unnecessarily (use `useMemo` if data is static) [15]. Check that images or data lists use lazy loading or pagination for performance [16] [14].
- **State management:** Avoid lifting state unnecessarily high or duplicating state between components. If multiple components share the same data, consider `Context` or a global store instead of prop-drilling. Remove any unused `setState` calls or variables (ESLint can catch unused vars [2]). Use `useRef` for values that don't affect rendering (like timers or DOM nodes) to prevent extraneous re-renders [17].
- **Asynchronous calls:** If multiple components fetch the same data separately, consider consolidating into a single request to reduce load. Debounce frequent calls (e.g. search inputs) to avoid spamming the backend. Use `React Query` or similar libraries to cache results and avoid repeated fetches on re-mount.
- **Layout optimization:** Use `React.Fragment` to group elements without extra DOM nodes, as per best practices [12]. Ensure CSS is efficient: avoid deep or overly specific selectors that slow down rendering. If images are static, serve them compressed or via a CDN.
- **Bundle size:** Remove unused libraries. For example, if the project has lodash but only uses one function, import just that function (or use native JS) to minimize bundle. Use code-splitting for large components (React `lazy()`), so that the initial load is fast [12] [18].
- **Linting and formatting:** Enforce consistent code style across the codebase. Run ESLint/Prettier and apply autofixes. This reduces "noisy" diffs and catches issues like missing semicolons or inconsistent quote usage. Consistency here aids maintainability [2].
- **Performance:** Use React Profiler or Chrome DevTools to identify any slow components. Memoize pure functional components to prevent re-rendering if props haven't changed. For example, wrap list items with `React.memo` if rendering large lists. Remove console logs or debugging code that affects performance (especially in production builds).

Missing Code Detection

- **Validation:** If any form inputs lack validation (e.g. numeric fields not checked, string lengths not enforced), flag these. Ensure all user data paths have checks. For example, if a date picker allows invalid dates, the app should prevent submission. If validation libraries like Yup/Formik can be used to simplify, suggest them.
- **Error handling:** Check for missing `try/catch` blocks around async calls. The app should handle API errors gracefully (e.g. display error toast). If any code assumes data exists without null checks, add conditionals or default values. Also suggest adding React Error Boundaries to catch unexpected errors and show a user-friendly message instead of breaking the app.
- **Documentation:** Note any missing comments or README updates. Each component should have a brief description. The code should include JSDoc or inline comments for non-trivial logic (e.g. scheduling algorithm). If no documentation, suggest adding one to help future developers understand the logic.
- **Unit and integration tests:** Flag absence of tests for critical features. Suggest writing Jest/RTL tests for form submission, state updates, and any pure functions (such as optimization logic). For example, a test could simulate adding a shift and verify it appears in the schedule. Ensure tests cover edge cases and are well-named [11].
- **Accessibility:** Check that images have `alt` text, form elements have associated `<label>`, and buttons have descriptive text or `aria-label`. Ensure color contrasts meet WCAG guidelines. If missing, add ARIA roles or labels as needed. Keyboard navigation should allow tabbing through all interactive elements.
- **Monitoring/logging:** If not present, suggest adding console error logging or analytics for catching unexpected issues in production.
- **Security features:** Verify presence of CSRF protection (if applicable), HTTP-only cookies for auth tokens, and no secret keys in source. If Supabase RLS or row-level security is planned (as hinted), confirm it's configured; otherwise note it as missing (see project TODOs if available).

Fixes and Improvements

For each issue identified, provide concise code diffs or snippets illustrating the fix. Address React-specific patterns and vibe-coding pitfalls:

- **Adding unique keys:**
  Before:

```
{items.map(item => <ListItem data={item} />)}
```

  After:

```
{items.map(item => (
  <ListItem key={item.id} data={item} />
))}
```

  Adding a stable `key` (e.g. `item.id`) prevents rendering issues [12].

- **Converting uncontrolled to controlled inputs:**
  Before:

```
<input type="text" name="email" onChange={handleChange} />
```

After:

```
<input
  type="email"
  name="email"
  value={formState.email}
  onChange={handleChange}
/>
```

Now the input's value comes from state, preventing React warnings and making validation possible.

- **Cleaning up effects:** If an effect uses timers or subscriptions:
Before:

```
useEffect(() => {
  const interval = setInterval(fetchData, 5000);
}, []);
```

After:

```
useEffect(() => {
  const interval = setInterval(fetchData, 5000);
  return () => clearInterval(interval);  // cleanup on unmount
}, []);
```

This avoids memory leaks by clearing the timer when the component unmounts.

- **Handling async errors:** Wrap API calls in try/catch:
Before:

```
const data = await supabase.from('shifts').select();
setShifts(data);
```

After:

```
try {
  const { data, error } = await supabase.from('shifts').select();
  if (error) throw error;
  setShifts(data);
} catch (err) {
  console.error('Failed to fetch shifts', err);
```

```
  // Show user-friendly error message
}
```

This ensures failed requests don't crash the app silently.

- **Adding input sanitization:** For any user HTML content:
  Before:

```
<div dangerouslySetInnerHTML={{ __html: post.content }} />
```

After:

```
import DOMPurify from 'dompurify';
// ...
<div
  dangerouslySetInnerHTML={{
    __html: DOMPurify.sanitize(post.content)
  }}
/>
```

Using DOMPurify prevents XSS attacks as recommended [9].

- **Refactoring duplicate code:** If multiple forms use similar fields, extract a reusable component:
  Before: (duplicated JSX in two places)

```
<input type="text" value={firstName} onChange={...} />
<input type="text" value={lastName} onChange={...} />
```

After: (create `TextInput` component)

```
import TextInput from './components/TextInput';
// ...
<TextInput label="First Name" value={firstName} onChange={...} />
<TextInput label="Last Name" value={lastName} onChange={...} />
```

This reduces redundancy and centralizes styling.

- **Optimizing re-renders:** Use `useMemo` for expensive calculations:

```
const optimizedSchedule = useMemo(() => computeOptimalSchedule(workers,
demand), [workers, demand]);
```

This ensures `computeOptimalSchedule` runs only when its inputs change, improving performance.

- **Simplifying event handlers:** Instead of inline arrow functions in JSX (which create a new function each render), define handlers outside render:
  Before:

```
<button onClick={() => handleDelete(id)}>Delete</button>
```

  After:

```
const onDelete = () => handleDelete(id);
// ...
<button onClick={onDelete}>Delete</button>
```

Apply each fix carefully and test the result. Each diff should be minimal yet effective. Ensure that the new code still compiles and passes linting.

Before committing changes as a PR: **Re-clone or re-fetch the latest repository**, check for merge conflicts, and resolve them. This avoids conflict issues in the PR. Double-check that all fixes comply with the project specifications in `agent.md`, and no required feature has been overlooked. Then create the commit and PR.

Final Validation and Testing

- **Manual testing:** Describe how you would simulate user interactions. For example, *"I would run the app locally (`npm start`), navigate to the scheduling page, input sample worker and demand data, submit the form, and verify that the shifts update correctly. I'd test edge cases like empty fields or extreme demand values. I'd click through all buttons (e.g. add, edit, delete shifts) and ensure proper feedback (loading spinners, success/error messages)."*
- **Automated tests:** Propose test cases. For instance, write a Jest test for the scheduling function: check that given 5 workers and demand of 10, the generated shifts match expectations. Use React Testing Library to simulate form submission and assert that the UI reflects the new data. Ensure tests run (e.g. `npm test`) and cover both happy paths and common failures. Aim for high coverage on critical logic [11].
- **Lint and build:** Run `npm run lint` and fix any new warnings. Run `npm run build` to ensure production build succeeds without errors. Fix any build issues (often minification errors or missing dependencies) before shipping. This step aligns with Codex advice to verify changes by running linters/builds [4].
- **Performance check:** Use React Profiler and DevTools to confirm there are no obvious bottlenecks in the main user flows. For example, verify that initial load time is reasonable and that no unnecessary re-renders occur during interactions.
- **Cross-browser and device check:** (If feasible) test on Chrome, Firefox, and a mobile device emulator to ensure the UI behaves consistently.
- **Review integration:** Finally, do a full code review of the updated codebase. Ensure that all parts fit together: the UI displays updated data correctly after API calls, styling remains consistent, and no new errors are introduced. Confirm that the pull request contains a clear description of changes and references to the issues addressed.

Note: Before finalizing the PR, **re-run all tests and linters** one more time to catch any final issues. Ensure that your commit message follows the project's style and clearly describes the changes.

Remember to verify compliance with all requirements in `agent.md` and that the final code is clean, efficient, and user-friendly [9] [4].

---

[1] [2] [9] [11] [12] [13] [14] [15] [16] [17] [18] React Code Review Checklist for Security & Performance Optimization | Redwerk
https://redwerk.com/blog/react-code-review-checklist-boost-security-performance/

[3] [4] [5] Prompting guide
https://developers.openai.com/codex/prompting/

[6] Don’t be a Vibe Coder. Problems with Vibe Coding | by Mehul Gupta | Data Science in Your Pocket | Medium
https://medium.com/data-science-in-your-pocket/dont-be-a-vibe-coder-30fa7c525971

[7] [8] [10] Top 5 problems with vibe coding | Glide Blog
https://www.glideapps.com/blog/vibe-coding-risks